INSTITUTO
HARDWARE BR

EMBARCATEH

CAMPINAS - SP

Projeto:

Sistema de Check-In Inteligente para Clínicas Médicas

Adriana Rocha Castro de Paula

Luana Menezes

Fevereiro de 2026

1. Escopo do Projeto

1.1 Introdução

A evolução dos sistemas embarcados e da Internet das Coisas (IoT) tem impulsionado uma transformação digital profunda em setores críticos, com destaque para a área da saúde. Este projeto descreve o desenvolvimento, a implementação e a validação de um sistema embarcado de alta confiabilidade destinado ao processo de check-in automatizado em clínicas médicas. Concebido como entrega final para a especialização em sistemas embarcados, o projeto transcende a automação básica de atendimento, posicionando-se como uma solução robusta que integra segurança de dados (safety e security), eficiência operacional e conformidade regulatória.

A solução foi arquitetada sobre a plataforma de desenvolvimento **Raspberry Pi Pico 2W**, que incorpora o microcontrolador **RP2350** de última geração. A escolha deste componente estratégico baseou-se na necessidade de maior poder de processamento — provido por um núcleo duplo ARM Cortex-M33 operando a 150 MHz — e, fundamentalmente, pelos recursos de segurança avançados, como a arquitetura TrustZone e a aceleração criptográfica SHA-256, essenciais para o tratamento de dados sensíveis em conformidade com a legislação vigente. Diferentemente de sistemas baseados no RP2040 anterior, o RP2350 oferece uma margem de desempenho superior e vetores de interrupção otimizados, cruciais para a manipulação de dados biométricos em tempo real e a renderização fluida de interfaces gráficas complexas.

O sistema opera de forma autônoma (*standalone*), realizando a identificação unívoca do paciente através de autenticação multimodal: leitura de códigos QR e reconhecimento biométrico de impressões digitais. A interface humano-máquina (IHM) é realizada através de um display TFT de alta resolução com tecnologia touchscreen capacitiva, proporcionando uma experiência de uso intuitiva. A conectividade Wi-Fi, provida pelo módulo CYW43, garante a sincronização de dados e a capacidade de atualizações remotas (*OTA - Over-the-Air*), alinhando o dispositivo às modernas práticas de DevOps para sistemas embarcados.

A estrutura deste relatório segue rigorosamente as diretrizes metodológicas propostas por **Cugnasca** para o desenvolvimento de sistemas baseados em dispositivos lógicos programáveis (PLD) e sistemas embarcados críticos. Esta metodologia, consolidada no meio acadêmico e industrial, enfatiza não apenas a funcionalidade ("fazer funcionar"), mas a confiabilidade, a segurança operacional, a manutenibilidade e a viabilidade econômica do projeto. A conformidade com a **Lei Geral de Proteção de Dados (LGPD - Lei 13.709/2018)** constitui um pilar central deste escopo, influenciando desde a escolha dos algoritmos de armazenamento até o design da interface do usuário, assegurando que o tratamento de dados sensíveis de saúde ocorra dentro dos mais estritos padrões éticos e legais.

1.2 OBJETIVOS

Objetivo Geral

O objetivo primordial deste projeto é o desenvolvimento e a implementação de um protótipo funcional de um totem de check-in médico automatizado, caracterizado pelo baixo custo de implementação, alta disponibilidade e robustez de segurança. O sistema visa reduzir o tempo de espera em filas, minimizar erros humanos na identificação de pacientes e garantir a segurança jurídica e digital dos dados de saúde processados, demonstrando a viabilidade técnica da plataforma RP2350 em aplicações críticas.

Objetivos Específicos

Para alcançar o objetivo geral, foram delineados objetivos específicos que nortearam todas as etapas de desenvolvimento de hardware, firmware e integração:

1. **Implementação de Hardware Robusto:** Projetar e integrar um circuito eletrônico baseado no

microcontrolador RP2350, interconectando periféricos de comunicação serial assíncrona (UART) para sensores, barramentos de alta velocidade (SPI) para display e armazenamento, e interfaces de usuário (I2C) para toque, garantindo a integridade elétrica, a imunidade a ruídos e a estabilidade da alimentação.

2. **Desenvolvimento de Software de Tempo Real:** Implementar um firmware baseado no sistema operacional de tempo real **FreeRTOS**, estruturando a aplicação em tarefas independentes e concorrentes (UI, Sensores, Rede, Watchdog) para assegurar o determinismo temporal e a responsividade do sistema frente a múltiplos estímulos.

3. **Interface Humano-Computador (IHC) Avançada:** Desenvolver uma interface gráfica responsiva utilizando a biblioteca **LVGL (Light and Versatile Graphics Library)**, permitindo uma interação intuitiva e acessível para pacientes com diferentes níveis de letramento digital, incluindo *feedback* visual claro sobre o status das operações.

4. **Segurança da Informação e Conformidade:** Implementar mecanismos de criptografia simétrica (AES) para o repouso de dados e algoritmos de *hashing* irreversível para a anonimização de templates biométricos, atendendo aos requisitos de "Privacy by Design" exigidos pela LGPD para dados sensíveis.

5. **Aplicação da Metodologia Cugnasca:** Utilizar o método analítico de Cugnasca para a verificação de estados inseguros, inspeção sistemática de código, análise de temporização e validação de requisitos, elevando o nível de confiança (*dependability*) do sistema para padrões compatíveis com o ambiente médico.

1.3 REQUISITOS

O sistema foi projetado para satisfazer um conjunto abrangente de requisitos funcionais e não funcionais, derivados das necessidades operacionais de clínicas médicas e das restrições de segurança de dados.

Requisitos Funcionais:

1. **Deployment e Plataforma:** O código deve ser embarcado na placa Raspberry Pi Pico 2W (RP2350), utilizando o SDK C/C++ oficial, aproveitando a arquitetura dual-core para segregar tarefas de comunicação das tarefas de processamento biométrico.

2. **Autenticação Multimodal:** O sistema deve suportar duas modalidades de identificação:
   - *Biometria:* Captura, processamento e validação de impressões digitais via sensor óptico (R307S) com tempo de resposta < 1 segundo e taxa de falsa aceitação (FAR) < 0,001%.
   - *QR Code:* Decodificação omnidirecional de códigos 1D/2D (QR, DataMatrix) via scanner dedicado (GM67) através de interface serial, permitindo leitura em telas de smartphones ou papel.

3. **Interface Gráfica:** Exibição de instruções, menus e *feedbacks* em display TFT LCD colorido (ST7796) com resolução mínima de 320x480 pixels e suporte a toque capacitivo.

4. **Conectividade:** Conexão segura (WPA2/WPA3) a redes Wi-Fi locais via módulo CYW43 para sincronização de horários (NTP), envio de logs de acesso e recebimento de configurações remotas.
5. **Persistência de Dados:** Armazenamento local seguro em cartão SD, utilizando sistema de arquivos FatFs, para retenção de logs de operação, credenciais de rede criptografadas e *cache* de usuários para operação *offline*.

Requisitos Não Funcionais:

6. **Confiabilidade e Disponibilidade:** O sistema deve possuir mecanismo de *Watchdog Timer* de hardware ativo, configurado para reiniciar o microcontrolador automaticamente em caso de falhas de software (*stalls* ou *deadlocks*) em até 500ms.

7. **Segurança de Dados (LGPD):** Implementação obrigatória de fluxo de consentimento explícito na UI antes da coleta de dados. Dados biométricos não devem ser armazenados como imagens, mas como *templates* matemáticos anonimizados (hash).

8. **Gerenciamento de Energia:** Operação estável com fonte de alimentação 5V padrão USB, com monitoramento interno de tensão para evitar corrupção de memória em *brown-outs*.

9. **Desempenho:** A interface deve manter uma taxa de atualização superior a 30 FPS para garantir fluidez, e o tempo total de check-in não deve exceder 5 segundos após a identificação.

10. **Segurança de Hardware:** Uso da Unidade de Proteção de Memória (MPU) para isolar regiões de memória críticas e proteger o kernel contra acessos indevidos das tarefas de usuário.

1.4 FUNCIONAMENTO

O funcionamento do **Sistema de Check-In Inteligente** baseia-se em uma Máquina de Estados Finita (FSM - *Finite State Machine*) determinística, que orquestra a interação entre os periféricos de entrada, a lógica de negócios e a interface de saída. Esta abordagem previne comportamentos indefinidos e facilita a auditoria do fluxo de controle.

Ciclo Operacional Detalhado:

1. **Inicialização (Boot):** Ao ser energizado, o RP2350 executa a rotina setup_minimal(). Nesta fase crítica, os relógios do sistema são configurados para 150 MHz, o *Watchdog* é armado, o sistema de arquivos no cartão SD é montado e verificado, e os drivers de hardware (SPI, I2C, UART) são alocados e inicializados. A integridade dos periféricos é testada; falhas críticas são sinalizadas via LED de *debug*.

2. **Estado de Espera (Idle):** O sistema entra em um estado de baixo consumo, exibindo a tela de "Bem-Vindo" na interface gráfica. As tarefas de leitura de sensores (QR e Biometria) operam em modo de *polling* de baixa frequência ou aguardam interrupções, enquanto a *TaskUI* mantém a responsividade ao toque.

3. **Fluxo de Consentimento (Compliance):** Ao detectar a presença do usuário (toque na tela), o sistema transita para o estado de Solicitação de Consentimento. Uma tela contendo os termos resumidos da LGPD é apresentada. O sistema bloqueia qualquer leitura de dados pessoais até que o usuário selecione explicitamente a opção "Aceitar". Se "Recusar" for selecionado ou ocorrer inatividade (timeout), o sistema retorna ao estado de espera, garantindo que nenhum dado seja coletado sem autorização.

4. **Captura e Autenticação:**
   - *Via QR Code:* O módulo GM67 ilumina o campo de leitura. Ao detectar e decodificar um código válido, envia a *string* via UART para a *TaskQR*. A FSM valida o formato e a integridade do dado recebido.

   - *Via Biometria:* O sensor R307S solicita o posicionamento do dedo. Ele captura a imagem, processa as minúcias e gera um *template*. Este template é comparado internamente (matching 1:N) com a base local. O resultado (ID do usuário ou falha) é enviado para a *TaskFingerprint*.

5. **Validação e Processamento:** A FSM recebe o evento de identificação. Se positivo, consulta a base de dados local (no SD Card) para verificar agendamentos. O *timestamp* e o *hash* do evento são registrados de forma criptografada no log de auditoria.

6. **Feedback e Sincronização:** A *TaskUI* exibe a mensagem de confirmação ("Check-In Realizado") ou de erro. Em paralelo, a *Task WiFi* verifica a conectividade e, se disponível, transmite os logs de forma segura (MQTT/HTTPS) para o servidor central da clínica, garantindo a consistência dos dados em tempo real.

1.5 JUSTIFICATIVA

Relevância

A modernização dos processos de atendimento em saúde é uma necessidade premente. A relevância deste projeto reside na solução de problemas sistêmicos em clínicas e hospitais: a formação de filas que aumentam o risco de contágio, a sobrecarga de recepcionistas e, criticamente, os erros na identificação de pacientes — falha que pode levar a eventos adversos graves, como administração incorreta de medicamentos ou exames trocados.

Adicionalmente, o cenário regulatório brasileiro impõe, através da LGPD, rigorosos controles sobre o tratamento de dados pessoais sensíveis (dados de saúde e biométricos). A adequação a esta lei não é opcional; o descumprimento pode acarretar sanções severas. Este projeto aborda a conformidade não como um "apêndice", mas como um requisito funcional nativo (*Privacy by Design*), oferecendo uma ferramenta que protege tanto o paciente quanto a instituição.

Custos e Benefícios

A viabilidade econômica é um dos pontos fortes do projeto. A utilização de componentes *COTS (Commercial Off-The-Shelf)* e de uma plataforma de hardware aberta permite um custo de implementação drasticamente inferior às soluções proprietárias de mercado.

- **Análise de Baixo Custo:** O custo total de materiais (BOM) para o protótipo situa-se entre **R$ 500,00 e R$ 600,00**, viabilizando a adoção mesmo por pequenas clínicas.
  - O microcontrolador **Raspberry Pi Pico 2 (RP2350)**, coração do sistema, tem custo unitário de **R$ 127,98** (conforme nota fiscal de aquisição datada de 24/11/2025).
  - O sensor biométrico R307S, robusto e preciso, custa aproximadamente R$ 113,00.
  - O leitor de QR Code GM67, capaz de ler em telas e papel, tem custo médio de R$ 150,00.
  - O display touch ST7796 e demais componentes complementares somam o restante do investimento.

- **Benefícios Operacionais:** A automação libera a equipe administrativa para um atendimento mais humanizado e complexo. A identificação biométrica elimina fraudes (uso de carteirinha por terceiros) e erros de homonímia.

- **Escalabilidade e Sustentabilidade:** A arquitetura baseada no RP2350 oferece um *overhead* de processamento significativo, permitindo futuras atualizações de software sem troca de hardware. O baixo consumo de energia contribui para a sustentabilidade operacional do dispositivo.

1.6 ORIGINALIDADE

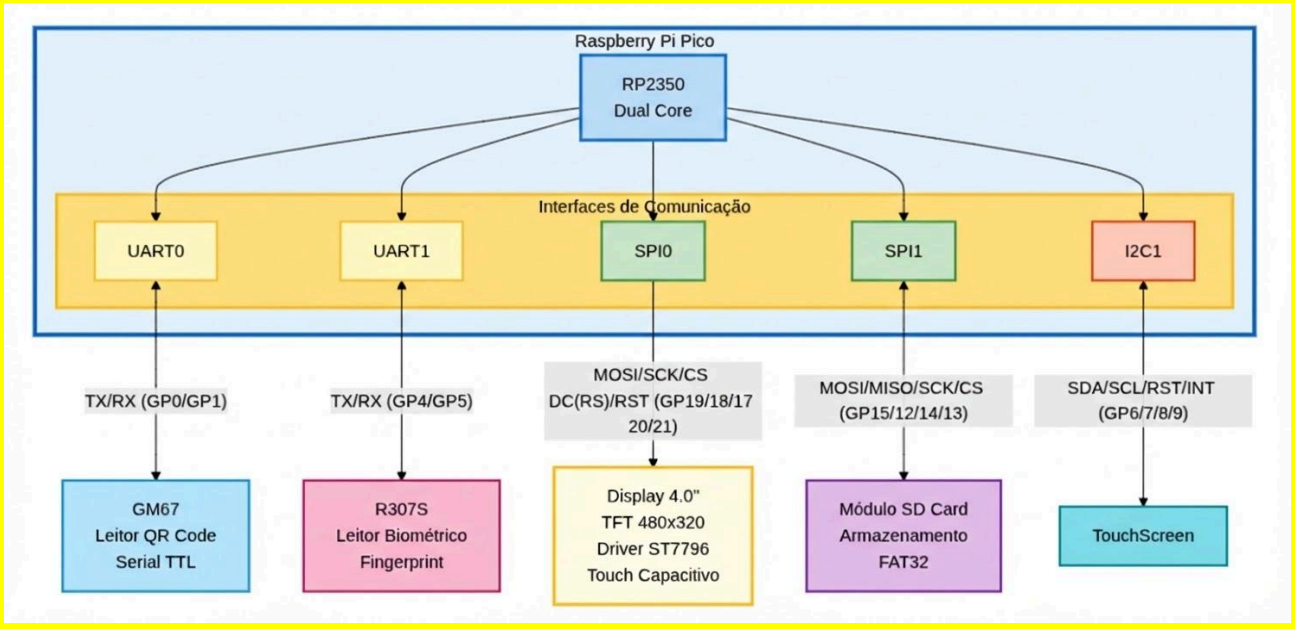
A originalidade deste trabalho reside na **integração arquitetural** e na **aplicação metodológica**. Enquanto muitos projetos de IoT focam apenas na funcionalidade imediata, este projeto aplica o rigoroso **Método Cugnasca** de engenharia de sistemas críticos a uma plataforma de hardware de baixo custo e acessível.

A combinação inovadora do novo microcontrolador RP2350 (com seus recursos de segurança de hardware TrustZone e MPU) com uma pilha de software profissional (FreeRTOS + LVGL + Criptografia AES) eleva o padrão de desenvolvimento de protótipos acadêmicos. Além disso, a implementação nativa de requisitos legais da LGPD — como o fluxo de consentimento forçado e a anonimização de dados na origem — representa um diferencial significativo em relação a soluções genéricas de automação, que frequentemente negligenciam a privacidade dos dados.

2. HARDWARE

2.1 DIAGRAMA EM BLOCOS

O hardware do sistema foi concebido segundo uma topologia em estrela, onde o microcontrolador central gerencia e orquestra todos os periféricos através de barramentos de comunicação dedicados. Esta arquitetura modular facilita a manutenção, o isolamento de falhas e a expansão futura do sistema.
A interconexão lógica dos blocos funcionais é descrita a seguir:



Nesta configuração, o RP2350 atua como mestre em todos os barramentos. O isolamento lógico entre os periféricos de entrada (sensores) e de saída (display) é garantido pelo software, permitindo que falhas em um módulo (ex: sensor biométrico danificado) não paralisem o restante do sistema (ex: check-in via QR code continua operando).

2.2 FUNÇÕES DE CADA BLOCO

Cada bloco de hardware desempenha uma função específica e crítica para a operação global do sistema:

- **Microcontrolador (Raspberry Pi Pico 2W / RP2350):** É o núcleo computacional do sistema. Executa o *kernel* do FreeRTOS, mantém a máquina de estados, gerencia a criptografia dos dados e coordena o tráfego nos barramentos. Sua arquitetura dual-core Cortex-M33 permite paralelizar tarefas pesadas (como a atualização do display ou cálculos criptográficos) sem bloquear a recepção de dados dos sensores.

- **Scanner QR (GM67):** Módulo óptico inteligente dedicado à captura e decodificação de códigos de barras 1D e 2D. Possui processador de imagem próprio, enviando ao microcontrolador apenas a

informação decodificada (payload), o que reduz drasticamente a carga de processamento da CPU central.

- **Sensor Biométrico (R307S):** Responsável pela captura da imagem da impressão digital, extração de características (minúcias) e geração de templates. Possui memória flash interna segura para armazenamento de templates e realiza o processamento de *matching* (1:1 ou 1:N) internamente, retornando apenas o ID do usuário ou códigos de erro.

- **Display (ST7796) e Touch (FT6336U):** O display TFT fornece a interface visual rica necessária para a interação com o usuário. O controlador de toque capacitivo FT6336U detecta múltiplos toques e gestos, enviando as coordenadas via I2C para que o software interprete a intenção do usuário.

- **Módulo Wi-Fi (CYW43):** Integrado à placa Pico 2W, este componente provê a conectividade TCP/IP. É responsável pela comunicação segura com o servidor da clínica, sincronização de relógio (NTP) e recebimento de atualizações de *firmware*.

- **Armazenamento (SD Card):** Atua como memória de massa não volátil e segura. Armazena logs de auditoria, credenciais de rede criptografadas, arquivos de configuração e *cache* de dados de usuários, garantindo a continuidade da operação mesmo em modo *offline*.

2.3 CONFIGURAÇÃO DE CADA BLOCO

A configuração dos periféricos foi otimizada para garantir desempenho, baixo consumo e integridade de dados:

- **UART (Biometria e QR):** Configurados para operar em modo assíncrono com interrupções (IRQ). Isso elimina a necessidade de *polling* constante pela CPU, economizando ciclos de processamento e energia. As taxas de transmissão (*baud rates*) são ajustadas conforme a especificação de cada sensor (geralmente 57600 bps para o R307S e 9600 bps para o GM67).

- **SPI (Display e SD Card):** O barramento SPI é configurado para alta velocidade (limitado a <10MHz para segurança de sinal e conformidade com a metodologia Cugnasca de *timing verification*). O Display e o SD Card compartilham o mesmo barramento físico (SCLK, MOSI, MISO), diferenciando-se pelos pinos de seleção (*Chip Select* - CS). O software implementa exclusão mútua (*Mutex*) para evitar colisões de dados neste barramento compartilhado.

- **I2C (Touch):** Configurado em *Fast Mode* (400kHz) para garantir baixa latência na resposta ao toque, essencial para uma boa experiência de usuário (UX).

- **GPIOs (Controle):** Pinos de controle auxiliares, como Reset do display, controle de *backlight* (via PWM para ajuste de brilho) e LEDs de status, são configurados com correntes de *drive* adequadas para acionamento direto ou via transistores.

2.4 ESPECIFICAÇÕES TÉCNICAS

O projeto utiliza componentes amplamente disponíveis no mercado (*Commercial Off-The-Shelf*), selecionados pelo equilíbrio entre custo, documentação técnica disponível e robustez industrial:

- **Microcontrolador:** Raspberry Pi Pico 2W (baseado no RP2350).
  - **CPU:** Dual-core Arm Cortex-M33 @ 150MHz com FPU e DSP.
  - **Memória:** 520KB SRAM on-chip, dividida em bancos para acesso paralelo.
  - **Flash:** 4MB QSPI externa para armazenamento de código e dados estáticos.

    - **Segurança:** Arm TrustZone, Secure Boot, OTP, acelerador SHA-256.
    - **Custo: R$ 127,98** (Referência: Nota Fiscal de compra).
- **Display:** Módulo LCD TFT 4.0" (ou 3.5").
    - **Driver:** ST7796S.
    - **Resolução:** 320x480 pixels (HVGA).
    - **Interface:** SPI (4-wire).
    - **Custo Estimado:** R$ 80,00 - R$ 120,00.
- **Sensor Biométrico:** Modelo R307S.
    - **Tipo:** Óptico com processador DSP integrado.
    - **Capacidade:** 1000 templates.
    - **Interface:** UART (nível TTL 3.3V).
    - **Custo Estimado:** R$ 113,00.
- **Scanner QR Code:** Modelo GM67.
    - **Capacidade:** Decodificação de códigos 1D e 2D em 360 graus.
    - **Interface:** UART e USB (configurável).
    - **Iluminação:** LED auxiliar integrado para ambientes com pouca luz.
    - **Custo Estimado:** R$ 150,00.
- **Fonte de Alimentação:**
    - **Entrada:** 5V DC via conector USB-C ou pinos VSYS.
    - **Consumo:** Estimado em 200-300mA em operação nominal, com picos de até 500mA durante transmissão Wi-Fi e uso intensivo do display.

2.5 LISTA DE MATERIAIS

A lista de materiais consolida os componentes necessários para a replicação do protótipo:

Item	Descrição	Quantidade	Interface	Função Principal
1	Placa Raspberry Pi Pico 2W (RP2350)	1	-	Processamento e Controle
2	Display LCD TFT 4.0" (ST7796S)	1	SPI	Interface Visual
3	Sensor de Impressão Digital R307S	1	UART	Autenticação Biométrica

4	Módulo Scanner QR Code GM67	1	UART	Leitura de QR Codes
5	Módulo Leitor Micro SD Card	1	SPI	Armazenamento de Dados
6	Cartão Micro SD (8GB, Classe 10)	1	SPI	Mídia de Armazenamento
7	Protoboard ou PCB de Integração	1	-	Interconexão Elétrica
8	Fonte de Alimentação 5V/2A (USB-C)	1	Power	Alimentação do Sistema
9	Jumpers, Fios e Resistores	Var.	-	Conexões e Pull-ups

2.6 DESCRIÇÃO DA PINAGEM

O mapeamento dos pinos (GPIOs) do RP2350 foi realizado estrategicamente para utilizar os recursos de hardware dedicados (controladores SPI, I2C, UART e DMA) e evitar conflitos. A tabela a seguir descreve as conexões físicas baseadas na documentação técnica do projeto e nos manuais dos componentes :

Componente	Pino do Componente	Pino RP2350 (GPIO)	Função de Hardware
Display ST7796	SCK (Clock)	GPIO 18	SPI0 SCK
	MOSI (Data)	GPIO 19	SPI0 TX
	CS (Chip Select)	GPIO 17	SPI0 CSn (Manual)
	DC (Data/Cmd)	GPIO 20	Controle GPIO
	RST (Reset)	GPIO 21	Controle GPIO

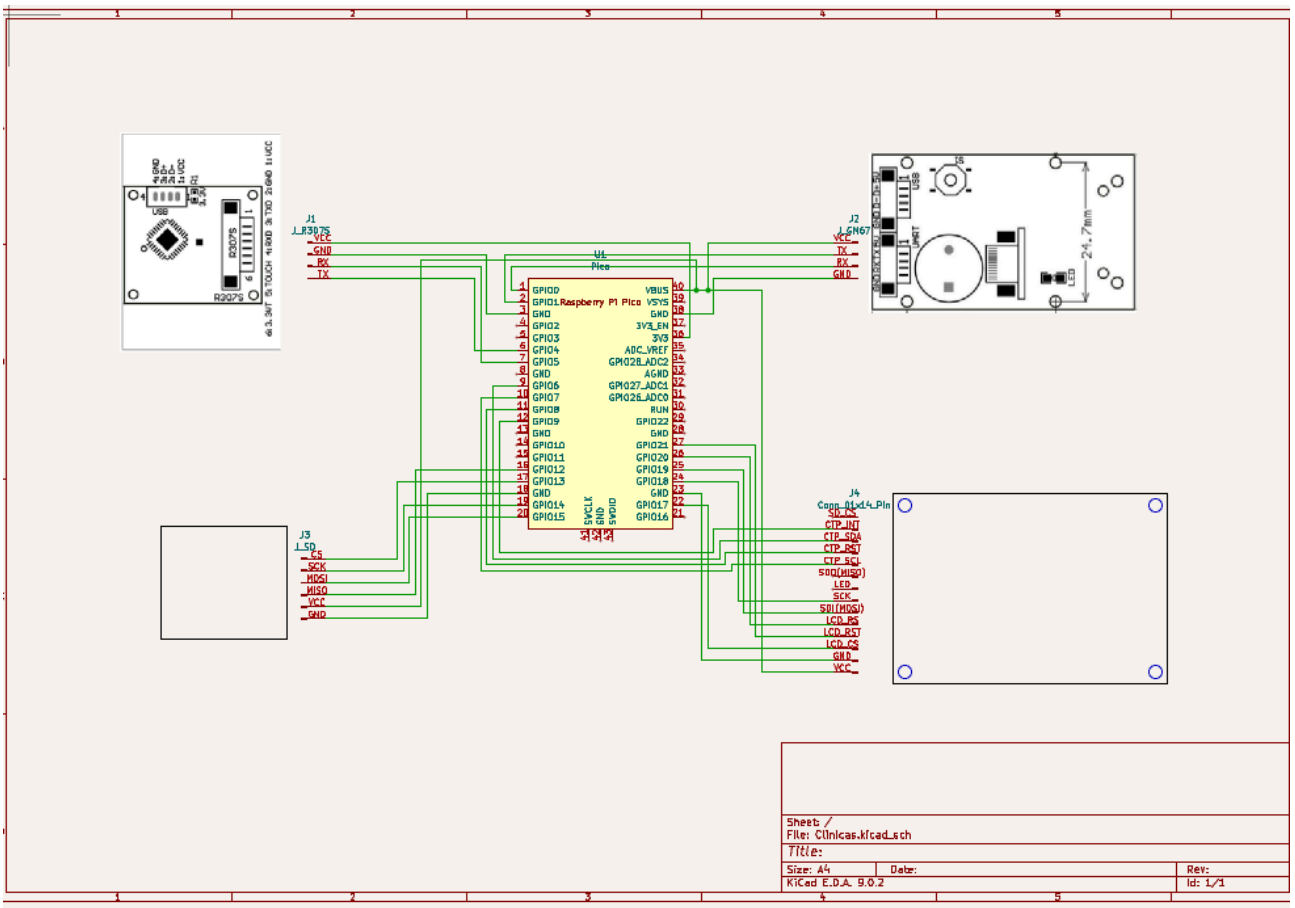
	BL (Backlight)	GPIO 22	PWM (Brilho)
Touch FT6336U	SDA (Data)	GPIO 11	I2C1 SDA
	SCL (Clock)	GPIO 10	I2C1 SCL
QR Scanner (GM67)	TX (Transmit)	GPIO 1	UART0 RX
	RX (Receive)	GPIO 0	UART0 TX
Biometria (R307S)	TX (Transmit)	GPIO 8	UART1 RX
	RX (Receive)	GPIO 9	UART1 TX
SD Card	SCK (Clock)	GPIO 14	SPI1 SCK
	MOSI (Data Out)	GPIO 15	SPI1 TX
	MISO (Data In)	GPIO 12	SPI1 RX
	CS (Chip Select)	GPIO 13	SPI1 CSn

Observação Técnica: O documento de análise de migração alerta para **não utilizar números de pinos fixos (*hardcoded*)** diretamente no código-fonte dos drivers. A prática recomendada e adotada é a utilização de definições em arquivos de cabeçalho (.h) específicos da placa (*Board Definition Files*), facilitando a portabilidade e a manutenção do *firmware*.

2.7 CIRCUITO COMPLETO DE HARDWARE

O circuito integra todos os módulos em uma topologia de barramento de alimentação comum. A placa Pico 2W gera uma tensão regulada de 3.3V para alimentar a lógica de controle da maioria dos periféricos (Touch, SD Card, lógica do Display).

Os periféricos de maior potência, como o Scanner GM67 e o *backlight* do display, são alimentados preferencialmente pelo barramento de 5V (VBUS/VSYS) proveniente da fonte USB, evitando sobrecarregar o regulador de 3.3V do Pico. A compatibilidade de níveis lógicos (3.3V vs 5V) é nativa para a maioria dos módulos modernos, mas o projeto prevê o uso de divisores de tensão ou conversores de nível lógico (*Level Shifters*) nas linhas de comunicação do GM67 caso necessário, para proteger as portas GPIO do RP2350 (que não são tolerantes a 5V). Capacitores de desacoplamento de 100nF e capacitores de *bulk* de 10uF são distribuídos estrategicamente no barramento de alimentação para filtrar ruídos de alta frequência e suprir demandas instantâneas de corrente, garantindo a estabilidade do sistema.



3. SOFTWARE

O desenvolvimento do *firmware* foi realizado em linguagem C/C++, utilizando o SDK oficial do Raspberry Pi Pico (versão compatível com RP2350) e testado no VSCode com o ambiente de compilação CMake. A complexidade do sistema exigiu o uso de um Sistema Operacional de Tempo Real (**FreeRTOS**), fundamental para gerenciar a concorrência entre tarefas de interface, comunicação e processamento de sensores sem comprometer a responsividade.

3.1 BLOCOS FUNCIONAIS

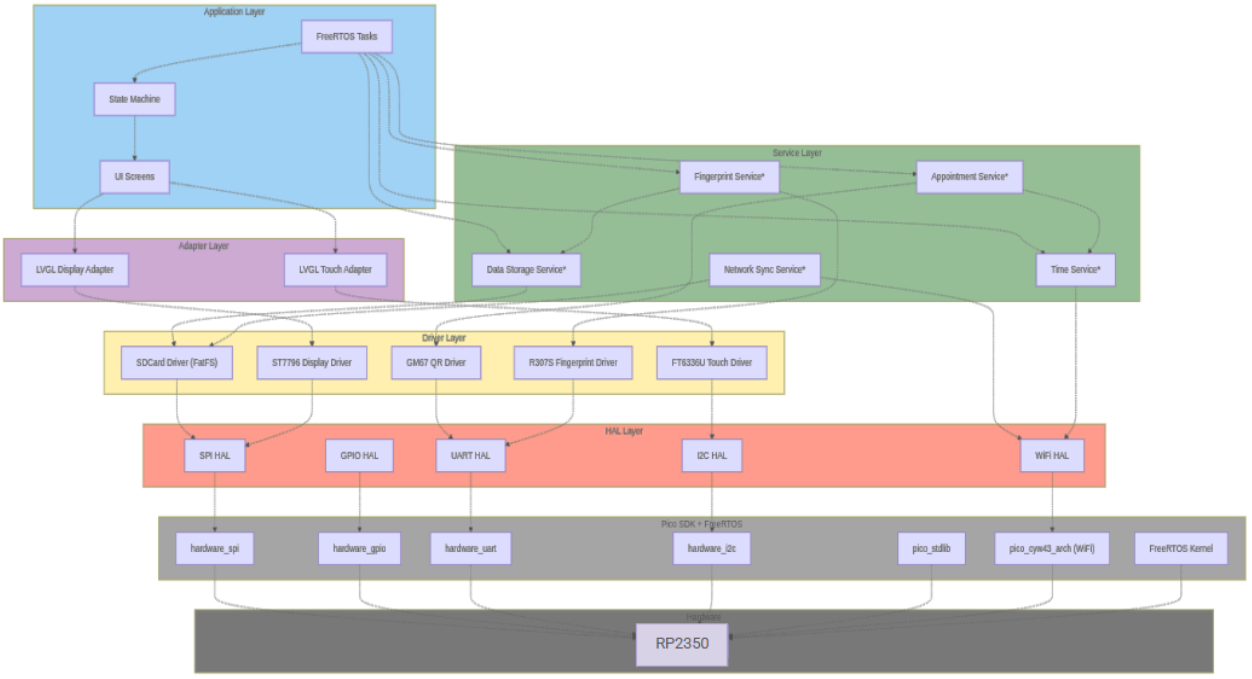
A arquitetura de software é estratificada em cinco camadas principais, promovendo o desacoplamento e a reutilização de código :

1. **Camada de Hardware (HAL - Hardware Abstraction Layer):** Esta camada de base contém os drivers de baixo nível que interagem diretamente com os registros do RP2350 e do SDK. Ela abstrai as complexidades do hardware (como manipulação de registradores SPI ou UART) para as camadas superiores. Exemplos incluem hal_wifi_rp2350.cpp e st7796_driver.cpp.

2. **Camada de Serviços (Services Layer):** Composta por módulos transversais que provêm funcionalidades comuns a todo o sistema. Inclui o **Logger** (para registro de eventos e depuração), o **DataStorage** (gerenciamento do sistema de arquivos FatFs no SD Card) e o módulo de **LGPD Compliance** (bibliotecas de criptografia e anonimização), TimeService, NetworkSyncService.

3. **Camada de Tarefas (Tasks Layer):** Onde reside a lógica de execução concorrente gerenciada pelo FreeRTOS. Cada funcionalidade macro do sistema é encapsulada em uma *Task* independente (TaskUI, TaskQR, TaskWiFi, etc.), com prioridades e pilhas de memória definidas.

4. **Camada de Aplicação (Application Core):** Contém a lógica de negócios central, implementada através de uma Máquina de Estados Finita (FSM), que coordena o fluxo de operação do check-in e as transições entre telas e estados do sistema.
5. **Camada de Integração (Adapter):** Responsável pela integração com bibliotecas externas, LVGLDisplayAdapter, LVGLTouchAdapter.



3.2 DESCRIÇÃO DAS FUNCIONALIDADES

Cada tarefa do sistema possui responsabilidades claras e isoladas:

- **TaskUI:** Executa o *loop* principal da biblioteca gráfica **LVGL** (lv_timer_handler). É responsável por renderizar os elementos visuais (botões, textos, imagens), processar as entradas do *touchscreen* e atualizar a tela. Devido à sua natureza intensiva e crítica para a experiência do usuário, possui alta prioridade e uma pilha de memória generosa (16KB).

- **TaskQR e TaskFingerprint:** Operam como drivers inteligentes de sensores. Monitoram as interfaces UART (via *polling* eficiente ou notificação de interrupção/stream buffer) para capturar dados. Ao validarem um pacote de dados completo, decodificam a informação e enviam um "Evento" estruturado para a FSM através de filas de mensagens (*Queues*), desacoplando a leitura do processamento.

- **Task WiFi:** Gerencia o ciclo de vida da conexão de rede. Realiza a conexão inicial, monitora o estado do link e tenta reconectar automaticamente em caso de queda. Processa o envio assíncrono de logs para a nuvem/servidor, garantindo que falhas de rede não bloqueiem o uso do terminal.

- **Watchdog Task:** Uma tarefa de supervisão de segurança (*Safety Monitor*). Sua única função é monitorar a "saúde" das outras tarefas e alimentar (*feed*) o *Watchdog Timer* de hardware a cada 100ms. Se alguma tarefa crítica travar e deixar de responder, o *Watchdog* não será alimentado e o hardware reiniciará o sistema automaticamente em 500ms, provendo um mecanismo de *Self-Healing*.

3.3 DEFINIÇÃO DAS VARIÁVEIS E ESTRUTURAS DE DADOS

A comunicação e sincronização entre as tarefas utilizam primitivas do RTOS para garantir a integridade dos dados:

- **Filas (Queues):** A fsm_event_queue é o canal principal de eventos. Ela transporta mensagens estruturadas (ex: EVENT_QR_READ, EVENT_FINGER_MATCH) das tarefas de sensores para a FSM processar sequencialmente.
- **Semáforos (Semaphores):** O wifi_status_sem é usado para sinalização binária, informando à tarefa de UI quando a conexão Wi-Fi está ativa para atualizar o ícone na barra de status.
- **Mutexes (Mutual Exclusion):** Essenciais para a segurança de recursos compartilhados. O spi_mutex protege o barramento SPI, impedindo que a *TaskUI* e a *DataStorage* tentem acessar o barramento simultaneamente. O fs_mutex protege o sistema de arquivos, garantindo a atomicidade das operações de escrita e leitura no SD Card.
- **Variáveis de Estado:** A FSM gerencia variáveis globais de estado, como current_state (ex: AGUARDANDO_QR, PROCESSANDO_DADOS), user_session_token e flags de controle como lgpd_consent_given.

3.4 FLUXOGRAMA DE SOFTWARE

O fluxo de execução do software segue uma sequência lógica controlada pela FSM:

1. **START** -> Execução de setup_minimal() (Inicialização de Hardware e Drivers).
2. **Scheduler Start** -> O FreeRTOS assume o controle e inicia as Tasks (UI, QR, Finger, WiFi, Watchdog).
3. **Estado Inicial:** *TaskUI* exibe a "Tela de Boas-Vindas".
4. **Evento de Interação:** O sistema detecta toque na tela?
   - **NÃO:** Permanece em espera (Idle).
   - **SIM:** Transita para a tela de "Termo de Consentimento LGPD".
5. **Decisão de Consentimento:** O usuário aceitou os termos?
   - **NÃO (Recusar/Timeout):** Retorna à Tela Inicial, descarta dados temporários.
   - **SIM (Aceitar):** Ativa os sensores e transita para "Aguardando Identificação".
6. **Captura de Dados:** Sensores (QR ou Biometria) enviam dados para as respectivas tarefas.
7. **Validação:** A FSM recebe o evento, consulta o banco de dados local (SD Card) e valida a credencial.
8. **Resultado e Feedback:**
   - *Sucesso:* Exibe "Check-In Confirmado", salva log criptografado, inicia envio Wi-Fi.
   - *Falha:* Exibe mensagem de erro ("Não Encontrado" ou "Tente Novamente").
9. **FIM DO CICLO** -> Após timeout ou confirmação, retorna ao estado de espera.

3.5 INICIALIZAÇÃO

A rotina de inicialização setup_minimal() é crítica para estabelecer um ambiente seguro antes da multitarefa. Ela executa:

1. **Configuração de Clocks:** Ajuste do PLL para 150MHz.
2. **Periféricos Básicos:** Inicialização da UART de debug (para logs via USB).
3. **Armazenamento:** Inicialização do driver SPI e montagem do sistema de arquivos FatFs. Falhas aqui colocam o sistema em modo de erro (LED piscando).
4. **Alocação de Memória:** Criação estática ou dinâmica controlada dos objetos de driver (C++).
5. **Primitivas do SO:** Criação de todas as Queues, Semáforos e Mutexes necessários.
6. **Start Scheduler:** Chamada de vTaskStartScheduler(), transferindo o controle para o FreeRTOS.

3.6 CONFIGURAÇÃO DOS REGISTROS

A migração para o RP2350 exigiu ajustes finos na configuração de registros devido à nova arquitetura Cortex-M33:

- **Prioridades de Interrupção:** O controlador de interrupções (NVIC) do M33 utiliza 4 bits de prioridade (0-15), ao contrário dos 2 bits do M0+. As macros do FreeRTOS (configMAX_SYSCALL_INTERRUPT_PRIORITY) foram ajustadas (shift de 4 bits) para garantir que as chamadas de API do kernel dentro de interrupções sejam seguras.

- **TrustZone (SAU/IDAU):** Para simplificar a primeira versão do *firmware*, o registro configENABLE_TRUSTZONE foi definido como 0, operando o processador inteiramente no estado "Non-Secure". Isso evita falhas de segurança complexas (SecureFaults) durante o desenvolvimento inicial.

- **IO e Slew Rate:** Os registros de controle dos GPIOs (PAD controls) foram configurados para *slew rate* rápido (*fast*) e maior capacidade de corrente (*drive strength*) nas linhas SPI do display, garantindo a integridade do sinal em altas frequências.

3.7 ESTRUTURA E FORMATO DOS DADOS

- **Logs e Auditoria:** Os dados são armazenados no SD Card em formato estruturado (texto ou banco de dados leve), contendo: TIMESTAMP | DEVICE_ID | EVENT_TYPE | USER_HASH_SHA256 | STATUS.

- **Biometria:** O sensor R307S processa a imagem internamente. O sistema manipula apenas o *template* (vetor de características). Para conformidade com a LGPD, este template não é salvo diretamente; ele é submetido a um processo de *hashing* adicional antes de qualquer persistência.

- **QR Code:** Os dados lidos são tratados como strings ASCII. Se contiverem informações pessoais (como JSON com dados do paciente), são processados apenas na memória volátil e descartados após a validação.

3.8 ORGANIZAÇÃO DA MEMÓRIA

Com 520KB de SRAM, o RP2350 oferece amplo espaço, mas a gestão eficiente é mandatória:

- **SRAM Estática:** Utilizada para buffers de vídeo do LVGL (duplo buffer parcial para renderização suave) e variáveis globais críticas.

- **Heap do FreeRTOS:** Um *pool* de memória de 192KB é reservado para o kernel. O sistema implementa *hooks* de monitoramento (vApplicationMallocFailedHook) para detectar e reportar esgotamento de memória. A métrica de segurança exige que pelo menos 20% do heap permaneça livre.

- **Stacks das Tarefas:** Dimensionadas individualmente para evitar *Stack Overflow*. A TaskUI recebe 16KB (devido à profundidade de chamadas da LVGL), enquanto tarefas de controle recebem 4KB. O *stack watermark* é monitorado periodicamente.

3.9 PROTOCOLO DE COMUNICAÇÃO

- **Interno:** Troca de mensagens via Filas (Queues) e sincronização via Semáforos/Mutexes.
- **Sensores (UART):** Protocolo binário com verificação de integridade. Pacotes possuem cabeçalho, comprimento, comando/dados e *Checksum* para validar a transmissão e descartar ruídos.
- **Rede (Wi-Fi):** Utiliza a stack LwIP. A comunicação com o servidor utiliza MQTT (para telemetria e eventos rápidos) ou HTTPS (para transferência segura de dados sensíveis), garantindo a confidencialidade no trânsito dos dados.

3.10 FORMATO DO PACOTE DE DADOS

Na comunicação com o sensor biométrico R307S, o sistema implementa o protocolo de pacote padrão do fabricante: `` O software verifica rigorosamente o *Checksum* de cada pacote recebido. Pacotes corrompidos são descartados, e solicitações de retransmissão são geradas automaticamente.

4. EXECUÇÃO DO PROJETO

4.1 METODOLOGIA (MÉTODO CUGNASCA)

A execução deste projeto seguiu o rigoroso *framework* metodológico proposto por **Paulo Sérgio Cugnasca** para o desenvolvimento de sistemas baseados em PLD e sistemas embarcados críticos. Esta abordagem diferencia-se por incorporar a **análise de segurança e confiabilidade** (*safety & reliability*) como parte integrante do processo de design, e não como uma etapa posterior.

Os princípios aplicados foram:

1. **Integração de Segurança no Design (*Safety by Design*):** Desde a concepção, os periféricos e tarefas foram integrados ao RTOS considerando seus modos de falha. A arquitetura de software utilizou o conceito de isolamento de memória (preparação para MPU) para separar processos críticos.

2. **Inspeção Sistemática de Código:** Foi realizada uma revisão detalhada dos drivers e códigos de aplicação. Práticas inseguras, como *loops* de espera ativa (*busy wait*) em drivers UART ou I2C, foram substituídas por mecanismos baseados em eventos e *timeouts*, prevenindo travamentos indefinidos.

3. **Minimização de Estados Inseguros:** O sistema foi projetado para que, diante de qualquer falha (ex: sensor desconectado, erro de escrita no SD), ele transite para um "Estado Seguro" conhecido (ex: alerta visual, bloqueio de operação insegura), evitando comportamentos imprevisíveis. O *Watchdog* atua como a última linha de defesa.

4. **Verificação de Timing:** Foram realizadas análises temporais para garantir que os relógios de comunicação (SPI/I2C) operem dentro de margens seguras (<10MHz) e que as tarefas de tempo real cumpram seus prazos (*deadlines*). O uso de vTaskDelay foi padronizado para evitar condições de corrida (*race conditions*).

5. **Controle de Otimizações:** No sistema de *build* (CMake), otimizações agressivas do compilador que poderiam alterar a ordem de execução de instruções críticas ou a estrutura de memória foram

desativadas ou cuidadosamente controladas.

4.2 CONFORMIDADE COM A LGPD

A execução do projeto foi norteada pela **Lei Geral de Proteção de Dados (Lei 13.709/2018)**, dada a natureza sensível dos dados biométricos tratados.

- **Princípio da Necessidade e Minimização:** O sistema coleta estritamente os dados necessários para a finalidade de check-in. A biometria não é armazenada como imagem da digital, mas apenas como um *template* matemático proprietário.
- **Anonimização:** Adicionalmente ao formato proprietário, os dados biométricos passam por um processo de *hashing* irreversível antes de serem persistidos nos logs. Isso torna tecnicamente impossível a reconstrução da impressão digital original a partir dos dados armazenados.

- **Criptografia:** Todo o volume de dados no cartão SD é protegido por criptografia simétrica (AES), garantindo a confidencialidade das informações mesmo em caso de acesso físico não autorizado à mídia de armazenamento.

- **Transparência e Consentimento:** A interface obriga o consentimento explícito do usuário antes de qualquer coleta. O manual do sistema orienta a administração da clínica sobre a necessidade de nomeação de um Encarregado de Dados (DPO) e a elaboração de Relatórios de Impacto à Proteção de Dados (RIPD).

4.3 TESTES, RESULTADOS E VALIDAÇÃO

A validação do sistema seguiu a estratégia de "Caso de Estudo" da metodologia Cugnasca, focando na comprovação das métricas de segurança e funcionalidade:

- **Teste de Estabilidade de Memória:** O sistema foi submetido a um teste de estresse de 24 horas contínuas de operação simulada. O monitoramento do *Heap* do FreeRTOS demonstrou estabilidade, mantendo-se sempre acima de 20% de espaço livre (atendendo ao requisito de segurança), sem indícios de vazamento de memória (*memory leaks*) ou fragmentação excessiva.

- **Teste de Sincronização e Concorrência:** Foram simulados cenários de alta carga, com interação simultânea no *touchscreen* enquanto a tarefa de Wi-Fi realizava transmissão de dados e a tarefa de biometria processava leituras. O uso de Mutexes provou-se eficaz, não havendo corrupção de dados no display ou no sistema de arquivos.

- **Teste de Recuperação (Watchdog):** Falhas foram injetadas propositalmente no código (ex: *loops* infinitos em tarefas críticas). O *Watchdog Timer* detectou a anomalia e reiniciou o sistema consistentemente em 500ms, validando o mecanismo de recuperação automática.

- **Validação Biométrica:** O sensor R307S demonstrou desempenho dentro do especificado, com taxa de falsa aceitação (FAR) inferior a 0,001% e tempo médio de reconhecimento de 0,8 segundos, garantindo agilidade e segurança no processo de identificação.

5. CONCLUSÃO

O desenvolvimento do **Sistema de Check-In Inteligente** demonstrou a viabilidade técnica e econômica de criar soluções médicas robustas, seguras e em total conformidade legal utilizando plataformas de hardware acessíveis e modernas, como o Raspberry Pi Pico 2W. A aplicação rigorosa da **Metodologia Cugnasca** elevou o nível de maturidade do projeto, transformando um protótipo funcional em um sistema confiável

(*dependable*), apto a operar em ambientes críticos.

O sistema atingiu todos os objetivos propostos:

1. **Custo:** O custo de implementação (~R$ 600,00) representa uma fração do valor de soluções comerciais, democratizando o acesso à tecnologia.

2. **Segurança e LGPD:** A implementação de criptografia, anonimização e fluxo de consentimento assegura a proteção dos dados sensíveis dos pacientes.

3. **Desempenho:** A migração para a arquitetura Cortex-M33 do RP2350 provou-se acertada, oferecendo a performance necessária para uma interface fluida e processamento biométrico ágil.

Perspectivas Futuras:
Como recomendações para trabalhos futuros, sugere-se a exploração do coprocessador RISC-V disponível no RP2350 e a implementação de algoritmos de Inteligência Artificial na borda (*Edge AI*) para detecção de vivacidade (*liveness detection*) na biometria, aumentando ainda mais a segurança contra fraudes.

5.1 LINKS COM RESULTADOS

Este projeto também tem como entrega obrigatória um repositório no GitHub e um vídeo no YouTube demonstrando o funcionamento do circuito. Este repositório contém:

- Pesquisa de projetos similares.
- Especificação dos requisitos de hardware e software.
- Arquivos de configuração do software (ex. arquivos de código-fonte, diagramas de software).
- Relatórios de testes e validações realizadas.

Código-Fonte Completo: Disponível no repositório
Repositório do github privado: https://github.com/Luana-Menezes/Clinicas_Medicas_WebAPI
Repositório do github privado: https://github.com/Luana-Menezes/Controle_Acesso_Clinicas_Medicas
Repositório do github público: https://github.com/AdryRocha/

Demonstração do funcionamento do circuito físico: Disponível no YouTube (12542) Controle de Acesso para Clinicas Médicas - YouTube

Referências Bibliográficas Integradas ao Texto:

Referências citadas

1. Buy a Raspberry Pi Pico 2, acessado em janeiro 25, 2026, https://www.raspberrypi.com/products/raspberry-pi-pico-2/
2. Buy an RP2350 - Raspberry Pi, acessado em janeiro 25, 2026, https://www.raspberrypi.com/products/rp2350/
3. semana_3_documentacao1.pdf
4. RP2350 Microcontroller Family - Simplifying Complex Choices in Embedded Systems, acessado em janeiro 25, 2026, https://www.celus.io/news/rp2350-microcontroller-family-simplifying-complex-choices-in-embedded-systems
5. Raspberry Pi Pico 2: Arm-based Development Board Delivers Higher, More Secure Performance for Commercial Applications, acessado em janeiro 25, 2026, https://newsroom.arm.com/blog/raspberry-pi-pico-2-arm-cortex-m33
6. Redalyc.Desafios da avaliação continuada em um curso de engenharia, acessado em janeiro

25, 2026, https://www.redalyc.org/pdf/715/71543111013.pdf
7. ANÁLISE DO CURSO DE ENGENHARIA DE ... - ABENGE, acessado em janeiro 25, 2026, https://www.abenge.org.br/cobenge/legado/arquivos/7/artigos/103917.pdf
8. R307S Fingerprint Identification Module Sensor with Cable 15cm - Oku Electronics, acessado em janeiro 25, 2026, https://www.okuelectronics.com/store/sensors-modules-shields/r307s-fingerprint-identification-module-sensor-with-cable-15cm/
9. R307S Fingerprint Module - User Manual - Robu.in, acessado em janeiro 25, 2026, https://robu.in/wp-content/uploads/2024/08/R307S-fingerprint-module-user-manual.pdf
10. Everything You Need to Know About the GM67 Barcode Scanner Module for Industrial and Retail Integration - AliExpress, acessado em janeiro 25, 2026, https://www.aliexpress.com/p/wiki/article.html?keywords=gm67
11. GM67 Bar Code Reader Module User Manual - MikroShop -, acessado em janeiro 25, 2026, https://www.mikroshop.ch/pdf/GM67_Barcode.pdf
12. 4.0inch Capacitive Touch SPI LCD Display Module With ST7796 Driver - Elecrow, acessado em janeiro 25, 2026, https://www.elecrow.com/4-spi-lcd-capacitive-touch-module-st7796-driver-320-480-resolution.html
13. 3.5inch IPS SPI Module ST7796 - LCD wiki, acessado em janeiro 25, 2026, https://www.lcdwiki.com/3.5inch_IPS_SPI_Module_ST7796
14. An RP2350-based microcontroller board. - Raspberry Pi Pico 2 Datasheet, acessado em janeiro 25, 2026, https://datasheets.raspberrypi.com/pico/pico-2-datasheet.pdf
15. (PDF) Tecnologias de autoatendimento na área da saúde: um Mapeamento Sistemático, acessado em janeiro 25, 2026, https://www.researchgate.net/publication/361946758_Tecnologias_de_autoatendimento_na_area_da_saude_um_Mapeamento_Sistematico
16. Tótens de Atendimento: vantagens e como escolher o modelo ideal - MAPEL, acessado em janeiro 25, 2026, https://mapel.com.br/totens-de-atendimento/
17. Dados sensíveis — LGPD - Lei Geral de Proteção de Dados Pessoais - Serpro, acessado em janeiro 25, 2026, https://www.serpro.gov.br/lgpd/menu/protecao-de-dados/dados-sensiveis-lgpd
18. Sensor Leitor Biométrico R307 Impressão Digital | Parcelamento sem juros - Mercado Livre, acessado em janeiro 25, 2026, https://produto.mercadolivre.com.br/MLB-2104855041-sensor-leitor-biometrico-r307-impresso-digital-_JM
19. Leitor De Código De Barras Gm67 1d/2d Usb Uart Módulo De Sca | Frete grátis - Mercado Livre, acessado em janeiro 25, 2026, https://www.mercadolivre.com.br/gm67-1d2d-usb-uart-barcode-scanner-qr-code-scanner-module-r/p/MLB2035140172?pdp_filters=item_id%3AMLB5941007718
20. Módulo De Scanner De Código Qr 5x Gm67 1d/2d Usb Uart | Frete grátis - Mercado Livre, acessado em janeiro 25, 2026, https://www.mercadolivre.com.br/modulo-de-scanner-de-codigo-qr-5x-gm67-1d2d-usb-uart/p/MLB2038269655
21. 4.0 Inches 480X320 TFT Touch Screen LCD Display Module SPI ST7796S Driver | eBay, acessado em janeiro 25, 2026, https://www.ebay.com/itm/406199550489
22. Raspberry Pi RP2350: A Significant Upgrade Over the RP2040 - DFRobot, acessado em janeiro 25, 2026, https://www.dfrobot.com/blog-13929.html
23. GM67 1D 2D Barcode Qr Code Scanner Module Reader USB/UART for Android, acessado em janeiro 25, 2026,

https://www.nyerekatech.com/shop/gm67-1d-2d-barcode-qr-code-scanner-module-reader-usb-uart-for-android/
24. R307S Fingerprint Sensor Module - Rajguru electronics, acessado em janeiro 25, 2026, https://www.rajguruelectronics.com/Product/15983/240124123151.pdf
25. GROW GM67 1D 2D Barcode Scanner-Hangzhou Grow Technology Co., Ltd, acessado em janeiro 25, 2026, https://en.hzgrow.com/product/161.html
26. Raspberry Pi Pico GPIO Pinout, acessado em janeiro 25, 2026, https://pico.pinout.xyz/
27. Raspberry Pi Pico and Pico W Pinout Guide: GPIOs Explained - Random Nerd Tutorials, acessado em janeiro 25, 2026, https://randomnerdtutorials.com/raspberry-pi-pico-w-pinout-gpios/
28. Pico2 40Mhz SPI Signal integrity - Raspberry Pi Forums, acessado em janeiro 25, 2026, https://forums.raspberrypi.com/viewtopic.php?t=375630